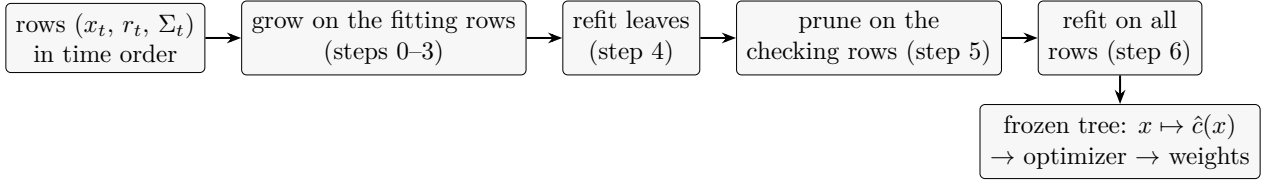


The SPO tree: model architecture and training pipeline

What the code does, step by step, with the equations

23 September 2026

This document describes the training of `SPOPortfolioTree` (`spo_tree.py`) as it is today, in the order the code runs it: the inputs, the decision problem the tree feeds, the loss, the replay, how a leaf score is fitted, how candidate thresholds are chosen (plain, quantile, refined, soft), how a split is accepted, how the tree is pruned and refitted, and how it is used. Every option added since the first version of the code is off by default, so with the defaults the training is the original one, plus one convention (Section 12). The experiments that test it are in `experiments_planted_threshold.tex`.



1 Inputs

Training sees T rows in time order, one per decision $t = 1, \dots, T$:

- $x_t \in \mathbb{R}^p$: the features known on the decision day;
- $r_t \in \mathbb{R}^n$: the simple returns of the n assets from this decision to the next ($2 \leq n \leq 4$);
- $\Sigma_t \in \mathbb{R}^{n \times n}$: a covariance of r_t estimated from the past only, positive definite.

Real data (`weekly_rows`): decisions at the close of a fixed weekday, $r_t = P_{d_t+7}/P_{d_t} - 1$, and $\Sigma_t = 7 \widehat{\text{Cov}}(\text{last } H \text{ daily returns}) + \varepsilon I$ with $H = 180$, $\varepsilon = 10^{-6}$; features are read on the decision day and must not use later data. The synthetic test (`planted_threshold_test.py`) builds the same rows with a decision every h days: $r_t = S_{t+h}/S_t - 1$, $\Sigma_t = \text{diag}(h\hat{\sigma}_t^2, \varepsilon)$ with the second asset cash. The last `test_periods` rows are held out; everything below sees the training rows only.

The decision problem has three parameters: the fee per traded notional $f \in \mathbb{R}_{\geq 0}^n$ (one value per asset), the risk aversion $\lambda > 0$ and the cap per asset $\bar{w}$. The holdings before the first decision are $u_1 = \mathbf{1}/n$ unless given.

Fitting and checking rows. With `validation_fraction` = $\rho_v > 0$ the training rows are cut in time order: the first $T_f = T - \text{round}(\rho_v T)$ are the *fitting rows*, on which the tree is grown; the last $T - T_f$ are the *checking rows*, which only prune it (step 5) and enter the final refit (step 6).

2 Features: what the tree needs

The code prescribes no feature; it reads the array x_t . What the training does with a feature decides what makes a good one.

- **Only the ordering matters.** A split is $x_j \leq \theta$ with θ between two observed values (Section 8); any increasing transformation (log, rank, scaling) gives the same partitions and the

same leaf scores. No normalisation is needed, and a threshold on a price *level* has no meaning out of sample: use stationary quantities (returns, ratios, volatilities, z-scores).

- **The model is piecewise constant:** a tree of depth D has at most 2^D leaves, hence 2^D score vectors. A feature should describe a *state* of the market, not a fine forecast; two nested splits capture one interaction (“calm and rising” against “calm and falling”), which is the case a tree represents and a one-feature rule cannot.
- **With fully invested weights only differences matter:** $w^*(c + \alpha \mathbf{1}) = w^*(c)$ (Section 3). With two risky assets a feature is useful to the extent that it predicts the *difference* of their returns; a cash column is what lets a feature reduce the exposure to the market.
- **Every feature adds candidate splits, and the best of many looks good by luck.** With N rows and a minimum leaf of m , each feature brings about $N - 2m$ thresholds; five features give over a thousand candidates at the root of a 15-year weekly fit, and on data with no signal the best of them still lowers the in-sample regret. Hence few features, each with an economic reason, and the quantile or soft options of Section 8 when the candidates must be tamed. A pure-noise column fitted separately is a cheap diagnostic.
- **Persistence sets the useful trading rate.** A feature that crosses its threshold often creates leaf changes, and each one can move the whole portfolio and pay the fee on both legs. The synthetic test (experiments report, Sections 4 and 8) gives the orders of magnitude: with a feature whose half-life is about a month, the threshold is recovered from daily to monthly decisions; with a half-life of a week, monthly decisions lose most of the value because the regime flips inside the holding period. Deciding more often than the feature moves adds fees, not information.
- **What can be found at all.** With 15 years of daily data and a volatility of 1% a day, a regime drift of $\pm 25\%$ a year is recovered reliably, $\pm 13\%$ a year about three times out of four and imprecisely, $\pm 5\%$ a year not at all. A feature whose edge is below a few basis points a day will not be learnt by this or any other method from that much history; test it with the planted-threshold protocol before trading it.

Practical candidates, to be validated on real data: momentum of each asset and of their ratio over one to three months, realised-volatility regime, drawdown from the recent high, and volume or funding-rate indicators if the data are reliable.

3 The decision problem

Given scores $c \in \mathbb{R}^n$, holdings u and a covariance Σ , the portfolio is

$$w^*(c; u, \Sigma) = \arg \max_{w \in \mathcal{W}} c^\top w - f^\top |w - u| - \lambda w^\top \Sigma w, \quad \mathcal{W} = \{w \in \mathbb{R}^n : 0 \leq w_i \leq \bar{w}, \mathbf{1}^\top w = 1\}. \quad (1)$$

It is solved exactly: at the optimum each weight is at 0, at $\bar{w}$, at u_i (the fee puts a kink there) or free on one side of u_i ; each of the 5^n patterns gives a small linear system, every admissible solution is feasible, and the best one is the unique optimum since the objective is strictly concave. All rows are solved in one vectorised call (`PortfolioOptimizer.solve`).

With two assets (asset a and b , holding v of a , $\Delta c = c_a - c_b$, $\phi = f_a + f_b$, $\kappa = 2\lambda(\sigma_a^2 + \sigma_b^2 - 2\sigma_{ab})$, minimum-variance weight $\bar{a}$ and $D = \Delta c - \kappa(v - \bar{a})$):

$$w_a = \begin{cases} v & |D| \leq \phi \quad (\text{no trade}), \\ \bar{a} + \frac{\Delta c - \phi \operatorname{sign}(D)}{\kappa} & \text{otherwise,} \end{cases} \quad \text{clipped to } [1 - \bar{w}, \bar{w}]. \quad (2)$$

So a leaf acts through one number Δc , the position is linear in it with slope $1/\kappa$, and the no-trade band has half-width ϕ . Only differences between scores matter: $w^*(c + \alpha \mathbf{1}) = w^*(c)$.

4 The loss: decision regret

The utility of a decision w in row t and the regret of scores c are

$$U_t(w) = r_t^\top w - f^\top |w - u_t| - \lambda w^\top \Sigma_t w, \quad (3)$$

$$\text{Reg}_t(c) = U_t(w^*(r_t; u_t, \Sigma_t)) - U_t(w^*(c; u_t, \Sigma_t)) \geq 0. \quad (4)$$

The first term is the best decision in hindsight from the same holdings u_t (a one-period oracle). The holdings u_t come from the replay of the current tree (Section 5) and are frozen while a round of the search runs. A row regret below -10^{-6} aborts the fit (solver check); values in $[-10^{-6}, 0)$ are set to 0.

5 The path and the Sharpe ratio

Scores $c_1, \dots, c_T$ are traded in time order from u_1 :

$$w_t = w^*(c_t; u_t, \Sigma_t), \quad \pi_t = r_t^\top w_t - f^\top |w_t - u_t|, \quad u_{t+1} = \frac{w_t \circ (1 + r_t)}{1 + r_t^\top w_t}. \quad (5)$$

Holdings drift between decisions, so fees are paid on every difference $|w_t - u_t|$. The Sharpe ratio of a path is $S = \text{mean}(\pi)/\text{std}(\pi)$ per decision (divisor $T - 1$, no risk-free rate); it is undefined with fewer than two rows, and it is 0 for a path with constant zero net returns (all cash, no trade), so that a split can be judged against an all-cash tree. The replay also returns every row's regret (4) along the path.

6 The tree and its score

A tree of depth at most D has internal nodes that split on one feature and leaves ℓ that hold one score vector $\hat{c}_\ell \in [-b, b]^n$ ($b = \text{prediction_bound}$). The scores are not forecasts: they are the inputs that make the optimizer take the right decision on the rows of the leaf.

Membership. A split node ν on feature j holds candidate thresholds $\theta_1 < \dots < \theta_K$ with weights $\pi_1, \dots, \pi_K$ summing to 1 (one threshold with weight 1 for a plain split). A row with feature value x goes left with weight

$$\mu_\nu(x) = \sum_{k: \theta_k \geq x} \pi_k \in [0, 1], \quad (6)$$

that is, the weight of the thresholds at or above x , and right with weight $1 - \mu_\nu(x)$. For a plain split $\mu_\nu(x) = \mathbf{1}\{x \leq \theta\}$. The weight of row t in leaf ℓ is the product of the memberships along the path from the root, $\omega_{t\ell}$; with plain splits every $\omega_{t\ell} \in \{0, 1\}$ and each row is in exactly one leaf. The score of row t is the mixture

$$\hat{c}(x_t) = \sum_{\ell} \omega_{t\ell} \hat{c}_\ell, \quad (7)$$

a piecewise-constant function of x for a plain tree, a continuous and monotone one between two leaves for a soft one. $I_\ell = \{t : \omega_{t\ell} > 0\}$ are the rows of leaf ℓ and $N_\ell = \sum_t \omega_{t\ell}$ its mass. A split node also reports one threshold, the π -weighted median of its candidates, and a *spread*, the distance between its 25% and 75% weight quantiles (0 for a plain split).

7 Fitting one leaf

With the holdings frozen, the score of leaf ℓ minimises its weighted regret:

$$\hat{c}_\ell \approx \arg \min_{c \in [-b, b]^n} L_\ell(c), \quad L_\ell(c) = \sum_{t \in I_\ell} \omega_{t\ell} \text{Reg}_t(c). \quad (8)$$

The search is deterministic (`_fit_leaf`): start from the ω -weighted mean of r_t over I_ℓ , clipped to $[-b, b]$; if a fallback vector is given, take it instead when it is strictly better (the fallback is the refitted parent for a child, the leaf's own vector for a refit, none for the root); then run 3 passes in which each coordinate in turn is moved to the best of 7 equally spaced values, the first pass over $[-b, b]$ and the next ones over a window of half-width $2b/6^k$ around the current best. One fit costs $2 + 3 \cdot 7n$ evaluations of L_ℓ , each a vectorised solve over the leaf.

8 Candidate thresholds

At a node with rows I and weights ω (mass $N = \sum \omega$), the minimum mass of a child is $m_* = \max(m, \lceil \varphi N \rceil)$ with $m = \text{min_samples_leaf}$ and $\varphi = \text{min_leaf_fraction}$. For feature j :

- (a) **Plain**. The candidates are the midpoints between consecutive distinct values of x_{tj} , $t \in I$, whose two sides both carry a mass of at least m_* . With `max_thresholds` = M set, an evenly spaced subset by rank of size M is kept.
- (b) **Quantiles** (`threshold_quantiles` = $(q_1, \dots, q_Q)$). Let $\theta_{(1)} < \dots < \theta_{(K)}$ be the admissible candidates of (a). Only the Q candidates at ranks $\text{round}(q_i(K-1))$, $i = 1, \dots, Q$, are kept: the quantiles of the admissible thresholds, evenly spread over what the leaf minimum allows and never on its boundary. *Why not the quantiles of the values.* The q -quantile of the node's values is the threshold that leaves a share q of the rows on the left; the rank rule leaves a share $\varphi^* + q(1 - 2\varphi^*)$, where $\varphi^* = m_*/N$ is the leaf minimum as a share of the node. The two coincide when there is no leaf minimum and differ only at the ends: with deciles and $\varphi^* = 0.1$ the values' rule puts its outermost candidates on the smallest leaves allowed (10% of the node), the rank rule 8 points inside (18%). The rank rule describes the search space itself, whatever the leaf minimum, and composes cleanly with the refinement (its windows stay inside the admissible range) and with the soft splits (whose average runs over candidates evenly spread over that range); the values' rule is recovered by choosing the quantile list accordingly.
- (c) **Refinement** (`refine_thresholds`). After the screen of (b), let θ_o be the best quantile candidate; every candidate of (a) strictly between the two quantile candidates neighbouring θ_o is screened as well. With plain splits the best of them, $\theta_\bullet$, replaces θ_o only if $Q(\theta_o) - Q(\theta_\bullet) > k \text{se}(Q(\theta_\bullet) - Q(\theta_o))$ with $k = \text{refine_standard_errors}$; with soft splits they simply join the candidate set.

The screen. Each candidate θ is scored cheaply: each child gets its clipped ω -weighted mean return as scores, and

$$Q(\theta) = \sum_{t \in I} \omega_t \text{Reg}_t(c_{\text{side}(t)}(\theta)), \quad c_{\text{left}}(\theta) = \text{clip}\left(\frac{\sum_{x_{tj} \leq \theta} \omega_t r_t}{\sum_{x_{tj} \leq \theta} \omega_t}, \pm b\right), \quad (9)$$

and likewise on the right. The standard error of a difference of two such sums is $\text{se} = \sqrt{|I|} \text{std}_t(\delta_t)$ with δ_t the per-row difference of the two regrets (rows treated as independent).

Plain splits (`smoothing` = 0): the `shortlist_size` candidates with the smallest Q over all features go to the full fit of step 1c; each has one threshold with weight 1.

Soft splits (`smoothing` = $\kappa > 0$). For each feature, all its candidates $\theta_1 < \dots < \theta_K$ are kept together with Boltzmann weights

$$z_k = \frac{Q(\theta_k) - \min_l Q(\theta_l)}{\text{se}(Q(\theta_k) - \min_l Q(\theta_l))}, \quad \pi_k = \frac{w(z_k/\kappa)}{\sum_l w(z_l/\kappa)}, \quad w(u) = e^{-u} \text{ or } e^{-u^2/2}, \quad (10)$$

the Laplace form e^{-u} by default and the Gaussian form with `smoothing_kernel`="gaussian" (at $\kappa = 1$ the two agree at $z = 2$, but the Gaussian form keeps more of the weight within one standard error of the best, 0.61 against 0.37, and cuts the tail beyond two, 0.01 against 0.05 at $z = 3$), so

that thresholds the data cannot tell apart from the best one (small z) share the weight and clearly worse ones get none ($z_k = 0$ when the regrets are identical, ∞ when they differ with zero standard error). The feature yields one candidate split, ranked by its best Q ; its children receive the rows with $\mu > 0$ on the left with weights $\omega\mu$ and the rows with $\mu < 1$ on the right with weights $\omega(1 - \mu)$, by (6). As $\kappa \rightarrow 0$ the weight concentrates on the best candidate and the split is the plain one.

9 Training, step by step

Algorithm (fit). Steps 0 to 4 see the fitting rows only.

0. **Root.** Fit one score vector on all fitting rows with (8), the holdings being $u_t = \mathbf{1}/n$ for every row. Replay the tree (5): this gives the holdings u_t the tree would really have had, the hindsight utilities of (4) and the Sharpe ratio S . Regrets are measured from these holdings until the next acceptance.
1. **For every leaf** ℓ of depth $< D$ and mass $N_\ell \geq 2m_*$:
 - a. *Parent refit.* Refit ℓ on the current holdings: $\hat{c}_\ell^{\text{refit}}$ (a yardstick and the children's fallback; never written into the tree).
 - b. *Candidates.* Build the candidate thresholds of every feature (Section 8), screen them with (9), and shortlist: the best `shortlist_size` thresholds (plain) or one weighted set per feature (soft).
 - c. *Children.* For each shortlisted split, route the rows with (6), fit both children with (8), and compute the regret reduction $\Delta = L_\ell(\hat{c}_\ell^{\text{refit}}) - L_{\text{left}}(\hat{c}_{\text{left}}) - L_{\text{right}}(\hat{c}_{\text{right}})$. Keep the split if $\Delta/N_\ell > \varepsilon_R$ (`min_regret_improvement`).
2. **Choice and confirmation.** Sort the kept splits of all leaves by Δ , largest first (SPOT's criterion). For the first one, rebuild the scores (7) of the tree with that split and replay it; accept it if $S' > S + \varepsilon_S$ (`min_sharpe_improvement`); otherwise try the next. If none passes, growth stops.
3. After an acceptance the replay of the new tree gives the new u_t , hindsight utilities and S ; return to step 1. The rows and weights of the two children are those of step 1c.
4. **Refit on the final holdings.** Refit every leaf with (8) (fallback: its own vector); keep the new vectors only if the replayed Sharpe ratio does not fall.
5. **Pruning on the checking rows** ($\rho_v > 0$), from the bottom of the tree up. For a split node ν : let $\mathcal{T}$ be the current tree (the split at ν with every surviving split below it) and $\mathcal{T}_{\nu \rightarrow \text{leaf}}$ the same tree with ν as a single leaf, its vector fitted with (8) on the fitting rows of ν (weights ω), from the holdings of $\mathcal{T}$ replayed on the fitting rows. Each tree trades the fitting rows, then the checking rows from the holdings it ends with. On the checking rows only,

$$\text{the split stays} \iff \bar{g} > \varepsilon_R \text{ and } \bar{g} > k_p \text{se}(\bar{g}) \text{ and } S^{\text{chk}}(\mathcal{T}) > S^{\text{chk}}(\mathcal{T}_{\nu \rightarrow \text{leaf}}) + \varepsilon_S, \quad (11)$$

where $g_t = \text{Reg}_t(\mathcal{T}_{\nu \rightarrow \text{leaf}}) - \text{Reg}_t(\mathcal{T})$ is the gain of the split on checking row t , $\bar{g}$ its mean over the checking rows and $\text{se}(\bar{g}) = \text{std}_t(g_t)/\sqrt{T - T_f}$ its standard error (a paired test, rows treated as independent); otherwise ν becomes that leaf. With the defaults $\varepsilon_R = 10^{-6}$, $\varepsilon_S = 0$ and $k_p = 0$ (`prune_standard_errors`) the test only asks for any improvement, which a split confirmed by a handful of noisy decisions passes; $k_p = 2$ asks for an improvement two standard errors above zero. With `prune_hurdle_from_depth` = D_p the hurdle $k_p \text{se}(\bar{g})$ applies only to splits at depth D_p or below, and the splits above it are judged by the plain rule; $D_p = 1$ spares the root split, which is rarely spurious, and targets the second-level leaves, where the noise splits live. A split is judged with everything below it; a split no checking row reaches cannot be confirmed.

6. **Refit on all training rows.** The rules are settled: all T rows are routed through the pruned tree with (6), the tree is replayed, every leaf vector is refitted with (8) on its rows (fitting and checking, fallback its own vector). This refit is always kept, since the vectors it replaces never saw the checking rows. Leaf masses N_ℓ then count all training rows.

Two remarks. Within a round, regrets are measured on the holdings of the tree *without* the candidate split; the change of path a split causes is seen by the Sharpe replay of step 2 and by the next round. And there is no single objective: leaf vectors minimise regret, the structure follows the largest regret reduction, the Sharpe ratio confirms on the fitting rows, the checking rows prune. Both tests of step 2 are in-sample; the pruning of step 5 is the out-of-sample judge. The logs `growth_log` and `pruning_log` record every accepted split and every pruning decision.

10 Using the tree

A frozen tree maps features to scores by (7) (`predict_returns`); the weights are $w^*(\hat{c}(x); u, \Sigma)$ with the current holdings and covariance (`predict_weights`). A backtest is the replay (5) of those scores (`replay`); `train_and_test` fits on all rows but the last `test_periods`, replays the training rows from $u_1 = \mathbf{1}/n$, then the test rows from the holdings the training replay ends with, against the same tree without splits and an equal-weight portfolio rebalanced every period.

11 Parameters

Parameter	default	run_real_data.py	note	role
max_weight $\bar{w}$	1	1	1	cap per asset; long-only, fully invested
fee_rate f	0.001	0.0003	(0.0003, 0)	fee per traded notional, scalar or per asset
risk_aversion λ	1	1	1	multiplies $w^\top \Sigma w$; positions are interior in a score window of width $2\lambda\sigma_h^2$
max_depth D	4	2	2	root is depth 0
min_samples_leaf m	10	20	20	minimum rows (mass) per leaf
min_leaf_fraction φ	0	0	0	minimum share of the node's mass per child
max_thresholds M	10	None	None	candidates per feature; None screens all
threshold_quantiles	None	None	None	quantile candidates only (by rank among the admissible thresholds), e.g. deciles
refine_thresholds	off	off	off	local refinement around the best quantile
refine_standard_errors k	0	0	0	hurdle of the refinement for plain splits
smoothing κ	0	0	0	soft splits with Boltzmann weights (10); 0 = plain
shortlist_size	3	3	3	screened splits per leaf that get a full fit (plain splits)
min_regret_improvement ε_R	10^{-6}	10^{-6}	10^{-6}	gate on Δ/N_ℓ in growth, on the mean checking regret in pruning
min_sharpe_improvement ε_S	0	0	0	margin on the Sharpe ratio, in growth and in pruning
prune_standard_errors k_p	0	0	0	pruning keeps a split only if its checking-row gain exceeds k_p standard errors
prune_hurdle_from_depth D_p	0	0	0	the hurdle applies to splits at depth D_p or below only
smoothing_kernel	laplace	laplace	laplace	form of the soft-split weights, e^{-u} or $e^{-u^2/2}$
validation_fraction ρ_v	0	0.25	0.25	last share of the training rows that prunes; 0 = no pruning
prediction_bound b	0.10	0.10	0.10	leaf vectors in $[-b, b]^n$
search_passes,	3, 7	3, 7	3, 7	leaf search
search_grid_size				
regret_tolerance	10^{-6}	10^{-6}	10^{-6}	numerical guard on row regrets

Table 1: The parameters: class defaults, the values of `run_real_data.py` (real data), and those of the experiments report (`planted_threshold_test.py`, base case). Section 8 of that report varies λ , f , D and the quantiles, Section 9 the split-search options, Section 10 the pruning hurdle.

12 What changed since the first version of the code

- **Sharpe ratio of an all-cash path.** It was NaN, so every comparison “with split > without” failed whenever the tree without the split sat in cash, and pruning removed such splits whatever

their merit. It is now 0 (Section 5).

- **Leaves carry weights.** Internally a leaf is (node, rows, weights, depth); with plain splits all weights are 1 and nothing changes. Candidate scores in steps 2 and 4 are rebuilt from the leaves rather than updated in place, which keeps the plain case exact.
- **Four options in the split search**, all off by default: quantile candidates with a minimum leaf fraction, local refinement with a standard-error hurdle, and soft splits with Boltzmann weights (Section 8). A soft node stores its candidates, their weights and a spread; its reported threshold is the weighted median.
- **A calibrated pruning hurdle**, off by default: the checking-row gain of a split must exceed k_p standard errors (Section 9, step 5); `pruning_log` records the standard error of every judged split. Since 23 September (evening), the hurdle can be restricted to splits at or below a given depth (`prune_hurdle_from_depth`), and the soft-split weights can take the Gaussian form (`smoothing_kernel`); this is version P4 of `model_history.md`.
- Everything else (decision problem, regret, path, leaf search, growth, pruning, final refit) is unchanged since the first version and is described above.